Heart Disease Risk Prediction

End-to-End MLOps Pipeline for Model Development, CI/CD, Deployment and Monitoring

Machine Learning Operations (MLOps) - Assignment 01

Prepared by: PRAVEEN T.(2024ad05233)

GITHUB link: https://github.com/praveenthangaraju99-ui/202405233_MLOPS_Assignment1

Video Explanation link: https://drive.google.com/file/d/16Xaq58v6qVEf2npWmXzDsxxdgM7PCjP9/view?usp=drive_link

Screenshots : Provided wherever required in this document. Also available in **\screenshots** folder

Problem Type	Binary classification - heart disease risk prediction
Dataset	UCI Heart Disease dataset loaded using ucimlrepo dataset id 45
Main Tools	Python, Pandas, Scikit-learn, MLflow, FastAPI, Docker, GitHub Actions, Kubernetes
Models Compared	Logistic Regression and Random Forest
Serving Endpoint	POST /predict with JSON input and confidence output
Monitoring	API logging and Prometheus-compatible /metrics endpoint

1. Executive Summary and Table of Contents

This project implements a complete MLOps workflow for predicting heart disease risk from clinical attributes. The pipeline starts with automated dataset acquisition from the UCI repository, performs exploratory analysis, builds reusable preprocessing steps, trains and compares two classification models, tracks experiments using MLflow, packages the best model, serves predictions through FastAPI, validates code with tests and CI/CD, containerizes the API using Docker, and provides Kubernetes and monitoring configuration for production-style deployment.

1.1 Table of Contents

1.2 Objectives Covered

- Acquire and clean the UCI Heart Disease dataset using the assignment-specified ucimlrepo method.
- Perform EDA with class balance, histogram, correlation heatmap and missing value plots.
- Build a reusable scikit-learn preprocessing and model pipeline.
- Train Logistic Regression and Random Forest models with cross-validation and hyperparameter tuning.
- Track experiments using MLflow and save model artifacts for reproducibility.
- Expose the final model through a FastAPI prediction service with monitoring endpoints.
- Include tests, GitHub Actions workflow, Dockerfile, Kubernetes manifests and monitoring configuration.

Prerequisite commands to run:
Setup environment: python -m venv mlops
Activate environment: .\mlops\Scripts\Activate.ps1
Install requirements:

1. **python -m pip install --upgrade pip setuptools wheel**
2. **pip install --prefer-binary -r requirements.txt**

2. Dataset Acquisition and Data Understanding

Command used: python src\data_acquisition.py

The dataset used in this project is the UCI Heart Disease dataset. The assignment specifies using the Heart Disease UCI dataset from the UCI Machine Learning Repository. In this implementation, the dataset is fetched programmatically using the ucimlrepo package, which makes the data acquisition step repeatable on a clean machine.

```
from ucimlrepo import fetch_ucirepo
heart_disease = fetch_ucirepo(id=45)
X = heart_disease.data.features
y = heart_disease.data.targets
```

The raw feature table and target table are combined into one dataframe. The original diagnosis target can contain multiple disease severity levels, so it is converted into a binary target for this assignment: 0 represents absence of heart disease and 1 represents presence of heart disease. This matches the binary risk prediction objective.

2.1 Data Files Generated

File	Location	Purpose
heart_disease_raw.csv	data/raw/	Stores the downloaded unmodified dataset.
heart_disease_clean.csv	data/processed/	Stores numeric cleaned data with binary target.
metadata and variables printout	terminal output	Provides dataset description and variable information for report reference.

2.2 Data Cleaning Decisions

- Question-mark values are treated as missing values and converted to pandas NA.
- All columns are converted to numeric format because the UCI file may load some columns as object/string type.
- The target is converted to binary using target > 0.
- Missing feature values are not manually dropped; they are handled inside the preprocessing pipeline to preserve reproducibility.

3. Exploratory Data Analysis

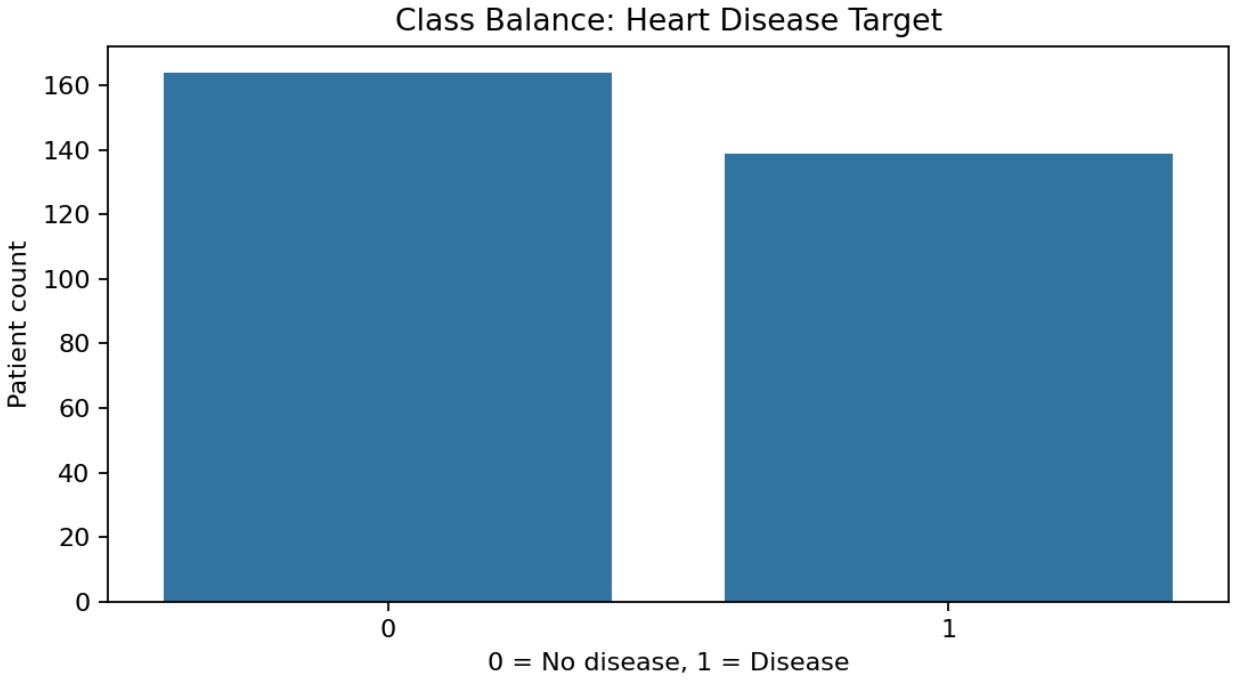
Command used: python src\eda.py

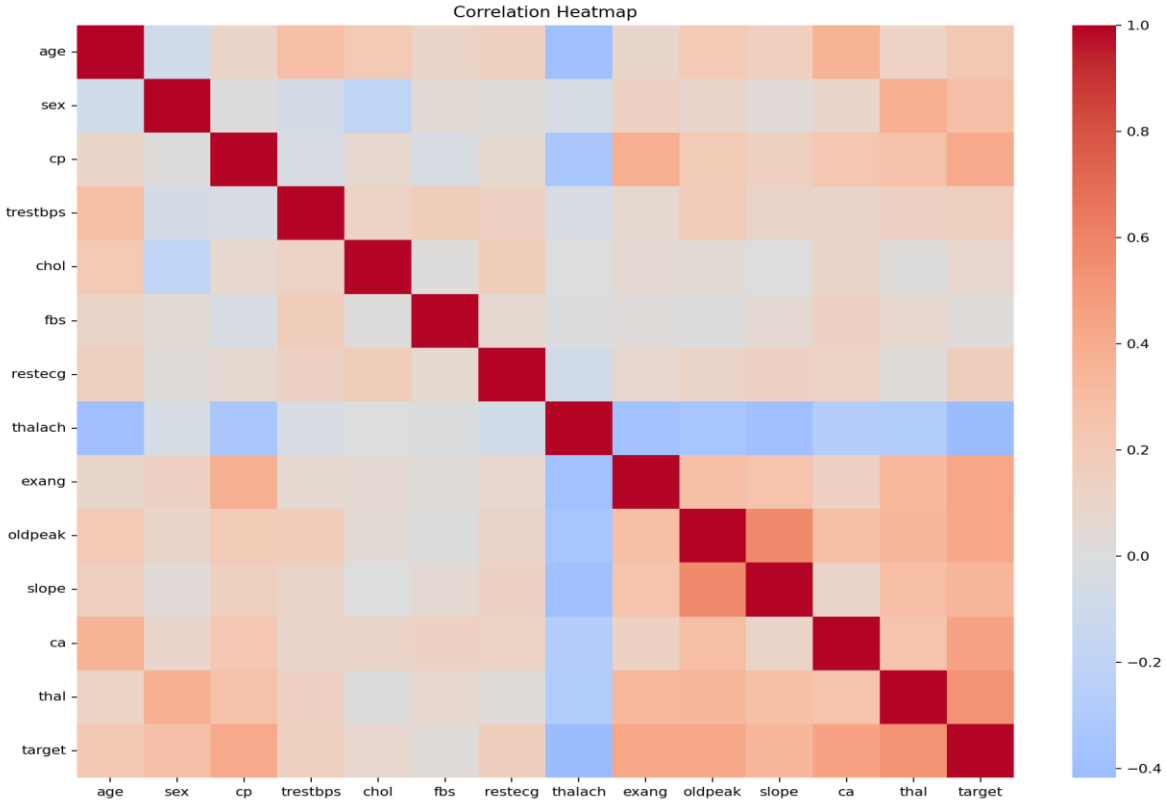
EDA is performed using Matplotlib and Seaborn. The objective of EDA is to understand the dataset before model training and to generate professional visual evidence for the assignment report. The script src/eda.py creates plots and saves them under docs/figures/.

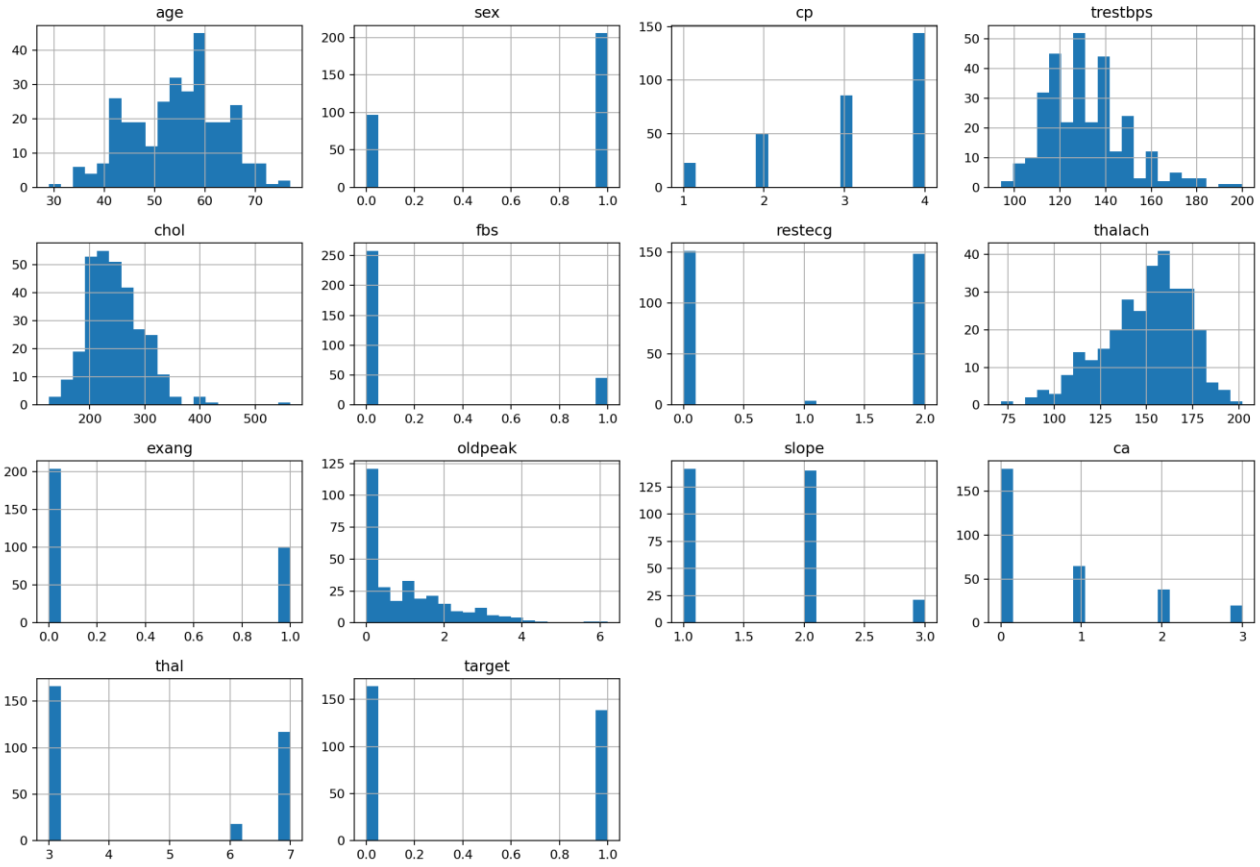
Plot	Output File	Reason for Inclusion
Class balance	class_balance.png	Shows whether no-disease and disease classes are balanced.
Histograms	histograms.png	Shows distributions of numeric clinical features.
Correlation heatmap	correlation_heatmap.png	Highlights linear relationships between features and target.
Missing values	missing_values.png	Shows columns requiring imputation.

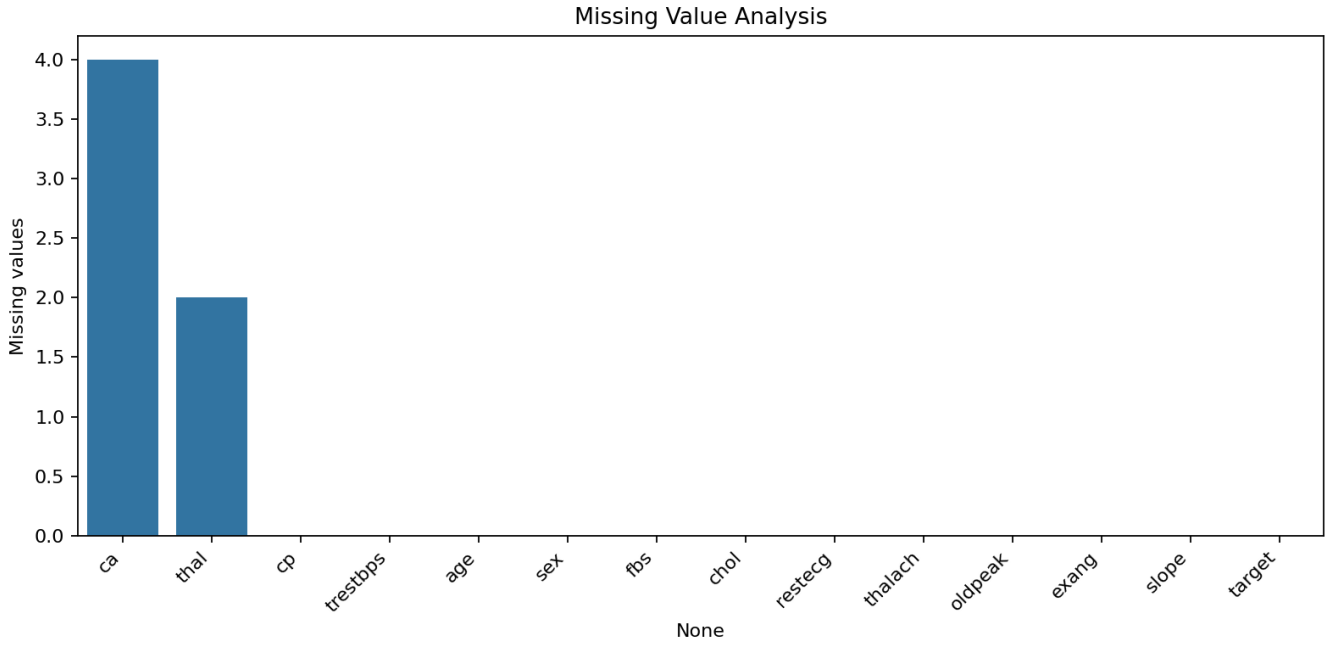
3.1 Key EDA Interpretation

The EDA plots are used to justify preprocessing and modelling decisions. Missing value analysis supports the use of imputation. Histograms show that numeric features have different scales, so scaling is useful for Logistic Regression. Class balance analysis helps decide whether class weighting should be tested during tuning. Correlation plots provide an initial indication of which clinical variables may be related to the heart disease target.




Correlation Heatmap
age
sex
cp
trestbps
chol
fbs
restecg
thalach
exang
oldpeak
slope
ca
thal
target
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4


age
sex
cp
trestbps
chol
fbs
restecg
thalach
exang
oldpeak
slope
ca
thal
target


Missing Value Analysis
Missing values
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
ca
thal
cp
trestbps
age
sex
fbs
chol
restecg
thalach
oldpeak
exang
slope
target
None

4. Feature Engineering and Preprocessing

Command used: python src\ preprocessing.py

The preprocessing stage is implemented using scikit-learn Pipeline and ColumnTransformer. This ensures that all preprocessing steps can be fitted during training and reused during inference without manual transformation. Keeping preprocessing and model together is important because incorrect feature processing during serving can lead to inconsistent predictions.

Feature Type	Columns	Transformation
Numeric	age, trestbps, chol, thalach, oldpeak	Median imputation + StandardScaler
Categorical	sex, cp, fbs, restecg, exang, slope, ca, thal	Most-frequent imputation + OneHotEncoder

5. Model Development

Command used: python src\ train.py

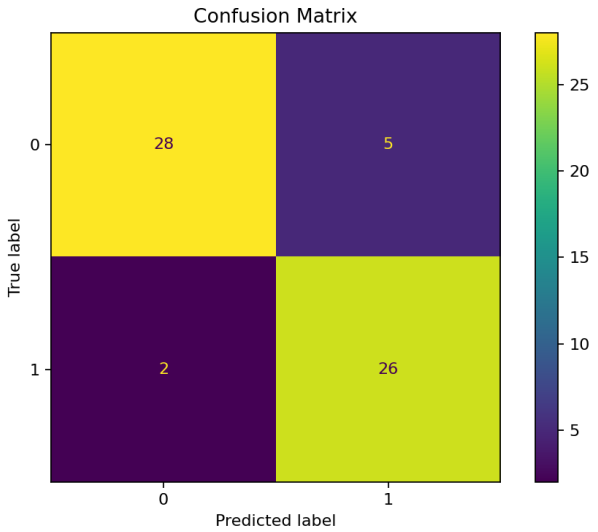
Two models are trained and compared. Logistic Regression is used as a baseline because it is interpretable and works well when feature scaling is done. Random Forest is used as a non-linear ensemble model because it can capture feature interactions and non-linear decision boundaries.

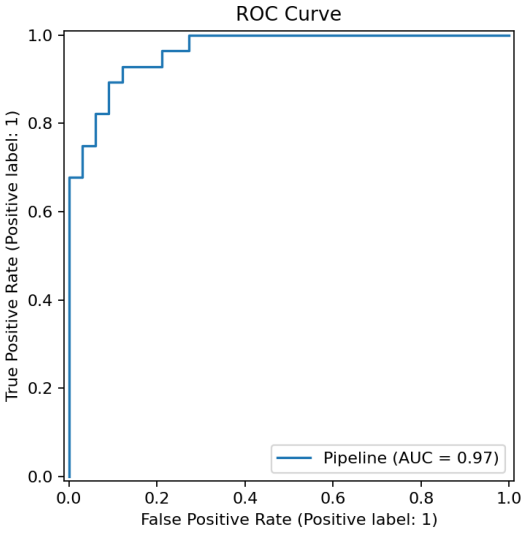
- Train/test split: stratified split with 20% test data.
- Cross-validation: StratifiedKFold with 5 folds.
- Hyperparameter tuning: GridSearchCV.
- Main tuning metric: ROC-AUC.
- Final evaluation metrics: accuracy, precision, recall, F1-score and ROC-AUC.

5.1 Model Tuning Grids

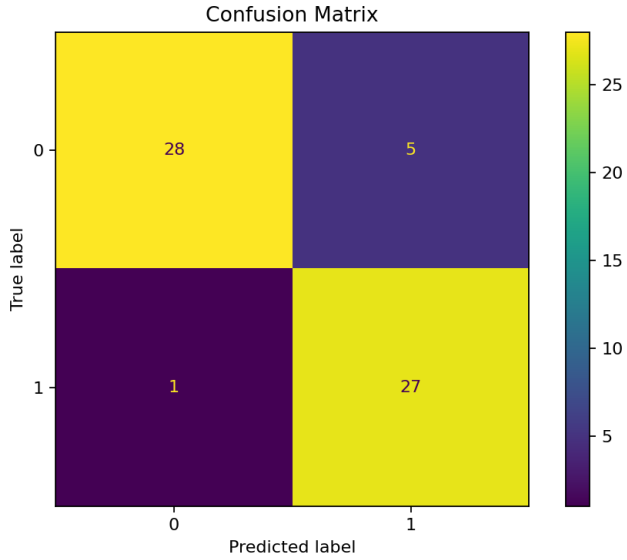
```
Logistic Regression: C = [0.1, 1.0, 10.0], class_weight = [None, balanced]
Random Forest: n_estimators = [100, 300], max_depth = [None, 4, 8], min_samples_split = [2, 5]
```

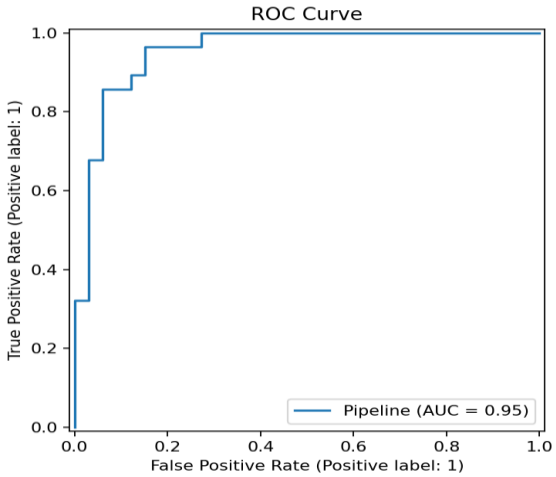
Logistic Regression:





Random Forest:





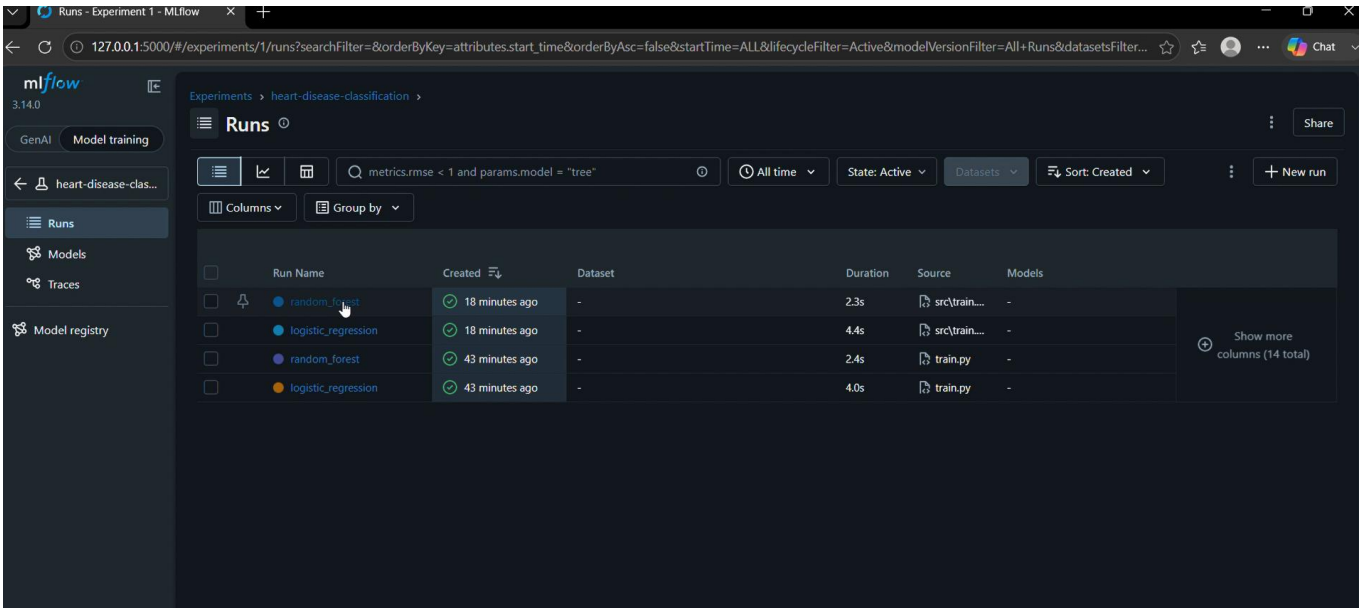
6. Experiment Tracking with MLflow

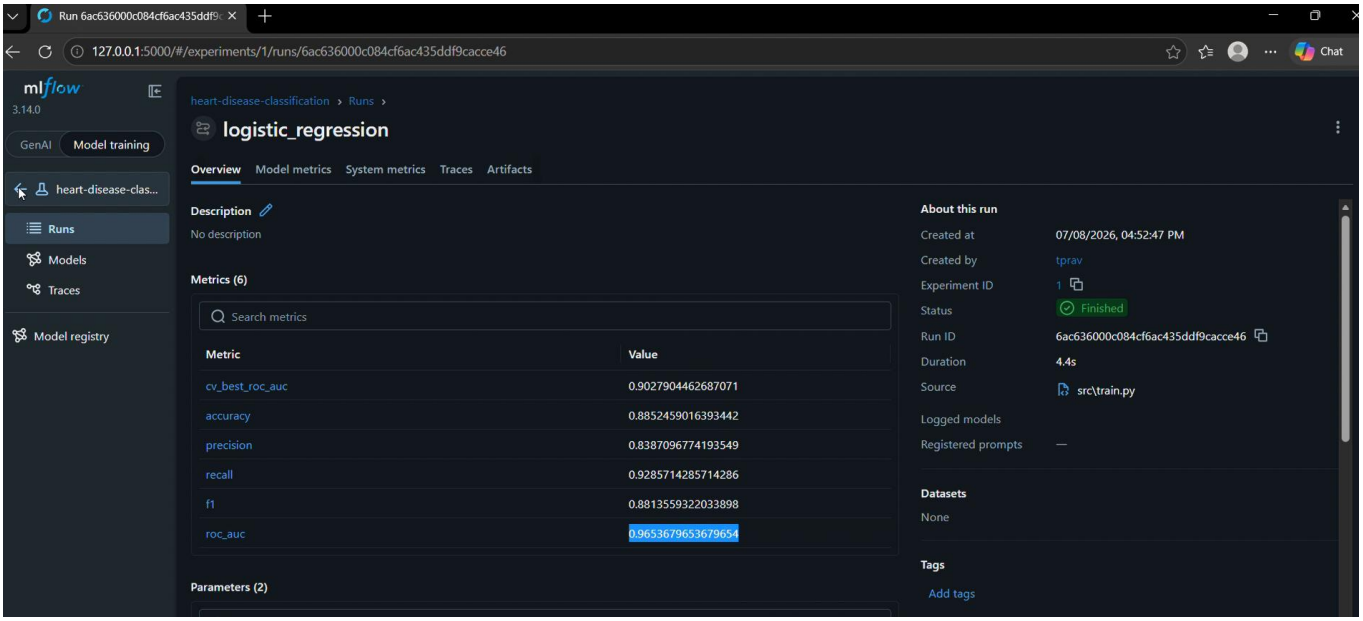
Command used: mlflow ui --backend-store-uri sqlite:///mlflow.db

MLflow UI: http://127.0.0.1:5000

MLflow is integrated for experiment tracking. In this project, SQLite is used as the MLflow backend because it works reliably on Windows with current Python and MLflow versions. Each model run logs parameters, cross-validation score, test metrics and visual artifacts.

MLflow Item	Logged Content
Parameters	Best GridSearchCV hyperparameters for each model
Metrics	CV ROC-AUC, accuracy, precision, recall, F1-score and test ROC-AUC
Artifacts	ROC curve, confusion matrix and joblib model artifact
Experiment name	heart-disease-classification

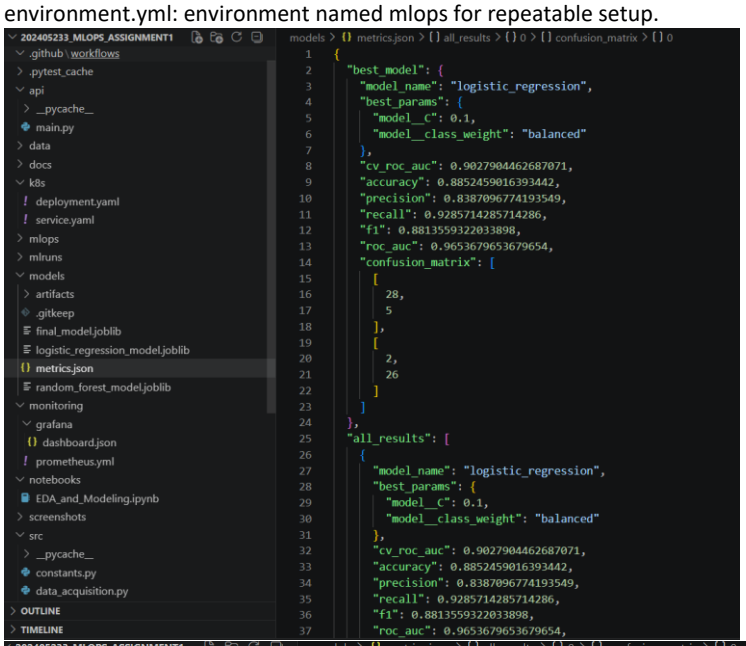


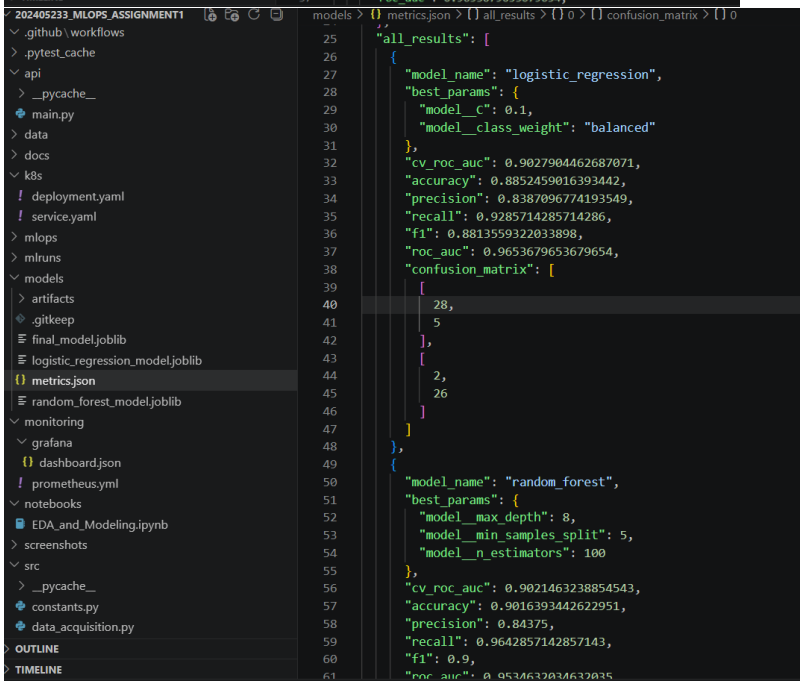


7. Model Packaging and Reproducibility

The best selected pipeline is saved as models/final_model.joblib. This object contains both preprocessing and the trained classifier. This is important because API inference loads the same pipeline used in training instead of manually reconstructing transformations. Reproducibility is also supported through requirements.txt and environment.yml.

- final_model.joblib: production inference model.
- metrics.json: selected model summary and comparison results.
- requirements.txt: Python dependency specification.
- environment.yml: environment named mlops for repeatable setup.





8. FastAPI Model Serving

Command used: uvicorn api.main:app --reload --host 0.0.0.0 --port 8000

Prediction test :

curl.exe -X POST http://localhost:8000/predict ` -H "Content-Type: application/json" ` -d "@sample_input.json"

Swagger UI: http://localhost:8000/docs

Health Endpoint: http://localhost:8000/health

Metrics Endpoint: http://localhost:8000/metrics

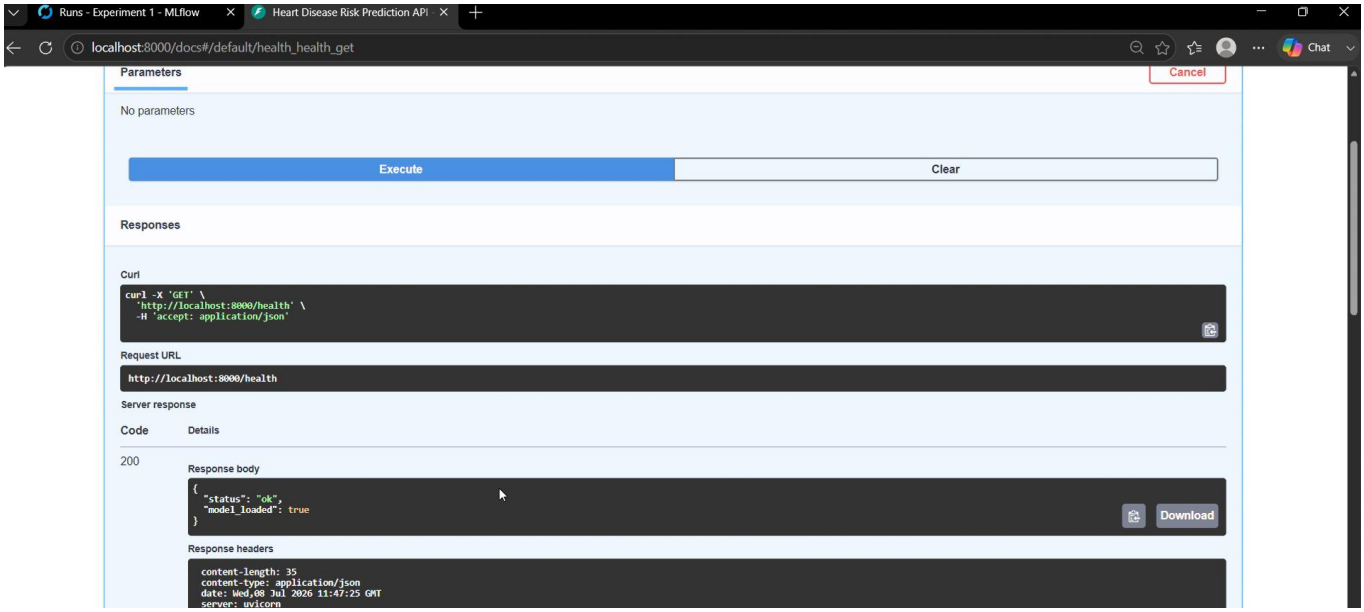
The final model is served using FastAPI. FastAPI was selected because it provides a clean API structure, request validation using Pydantic models and automatic Swagger UI documentation. The main service file is api/main.py.

Endpoint	Method	Purpose
/health	GET	Checks whether API is running and model is loaded.
/predict	POST	Accepts JSON patient features and returns class prediction and confidence.
/metrics	GET	Exposes Prometheus-compatible API metrics.

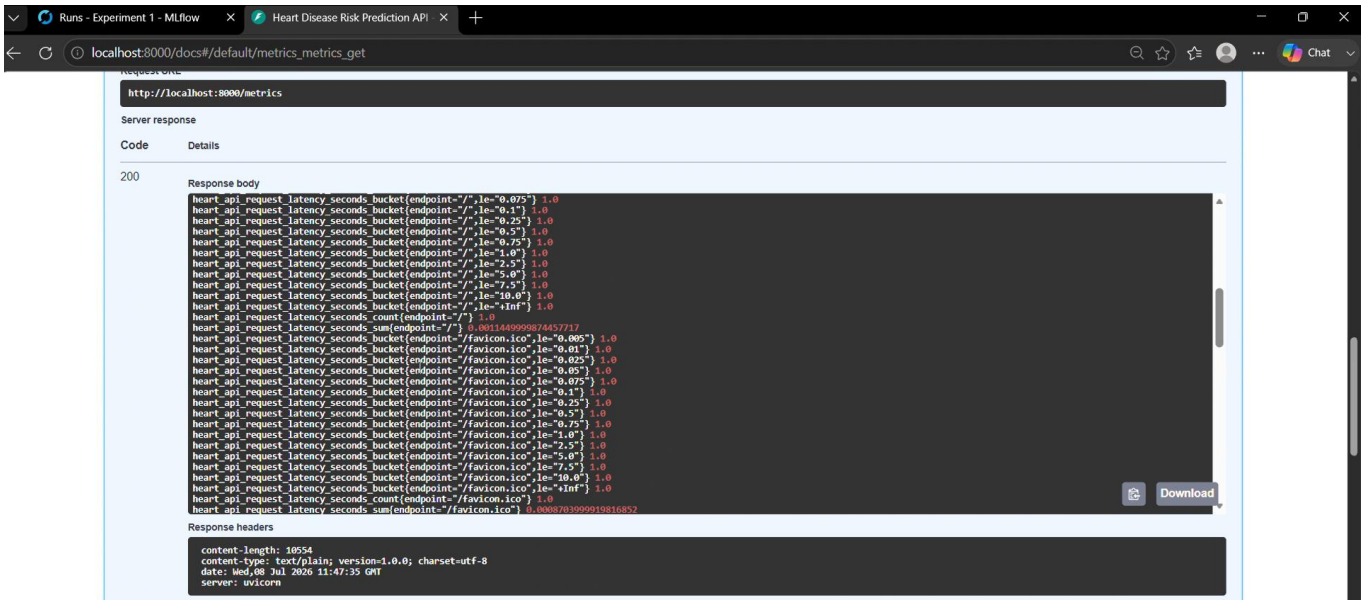
8.1 Sample Input

```
{
  "age": 63,
  "sex": 1,
  "cp": 3,
  "trestbps": 145,
  "chol": 233,
  "fbs": 1,
  "restecg": 0,
  "thalach": 150,
  "exang": 0,
  "oldpeak": 2.3,
  "slope": 0,
  "ca": 0,
  "thal": 1
}
```

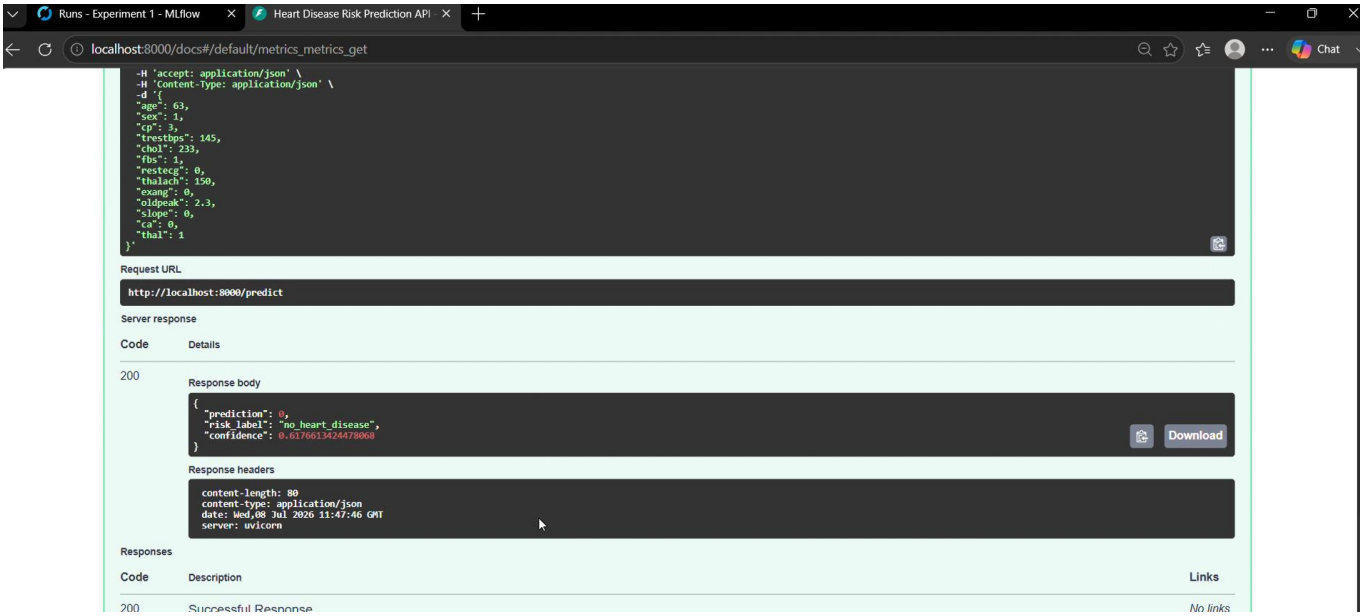
/health- GET



/metrics-POST



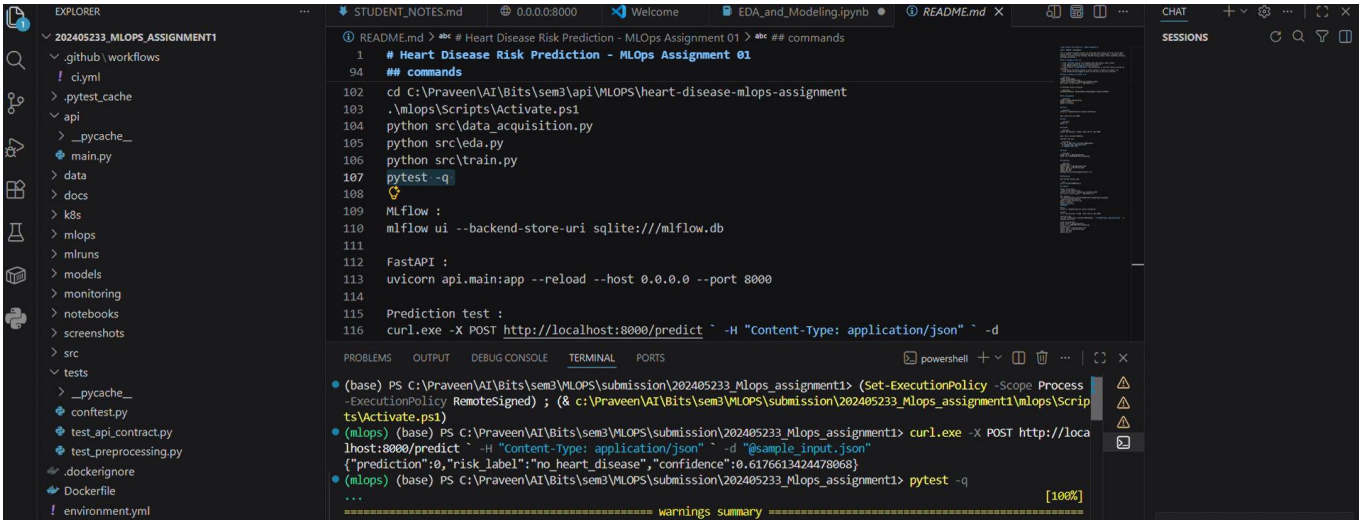
/predict-POST


Runs - Experiment 1 - MLflow
Heart Disease Risk Prediction API
localhost:8000/docs#/default/metrics_metrics_get
Chat
-H 'accept: application/json' \
-H 'Content-Type: application/json' \
-d '{
"age": 63,
"sex": 1,
"cp": 3,
"trestbps": 145,
"chol": 233,
"fbs": 1,
"restecg": 0,
"thalach": 150,
"exang": 0,
"oldpeak": 2.3,
"slope": 0,
"ca": 0,
"thal": 1
}'
Request URL
http://localhost:8000/predict
Server response
Code
Details
200
Response body
{
"prediction": 0,
"risk_label": "no_heart_disease",
"confidence": 0.6176613424478068
}
Download
Response headers
content-length: 80
content-type: application/json
date: Wed,08 Jul 2026 11:47:46 GMT
server: uvicorn
Responses
Code
Description
Links
200
Successful Response
No links

9. Automated Testing

Command used: pytest -q

Unit tests are included using pytest. The tests validate preprocessing behavior and the API health endpoint. These tests are also executed by the GitHub Actions workflow.



10. CI/CD Pipeline

The CI/CD workflow is implemented using GitHub Actions. The workflow is stored in .github/workflows/ci.yml and runs automatically on push or pull request to the main branch. It validates that the project can be installed, linted, tested and trained in a clean environment.

- Install dependencies using requirements.txt.
- Run flake8 linting to catch style and syntax issues.
- Run pytest unit tests.
- Execute data acquisition, EDA and model training as a smoke test.
- Upload model, EDA and MLflow artifacts for workflow traceability.

11. Docker Containerization

Command used:

docker build -t heart-disease-api .

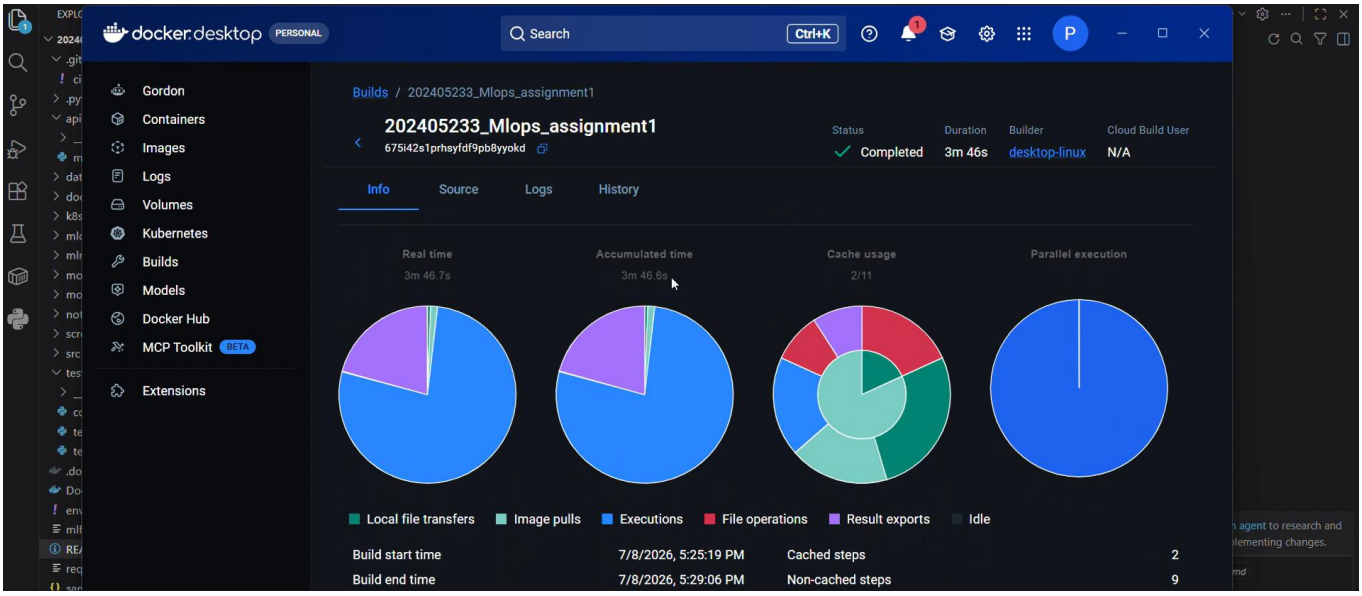
docker run -p 8000:8000 heart-disease-api

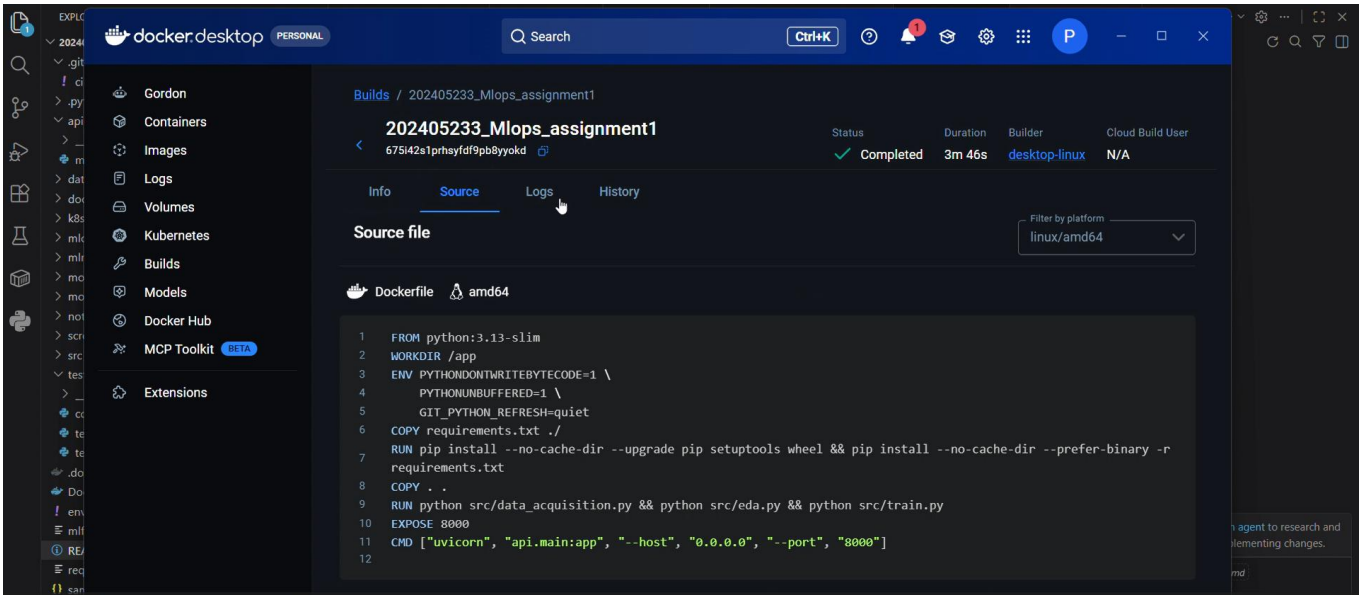
http://localhost:8000/metrics

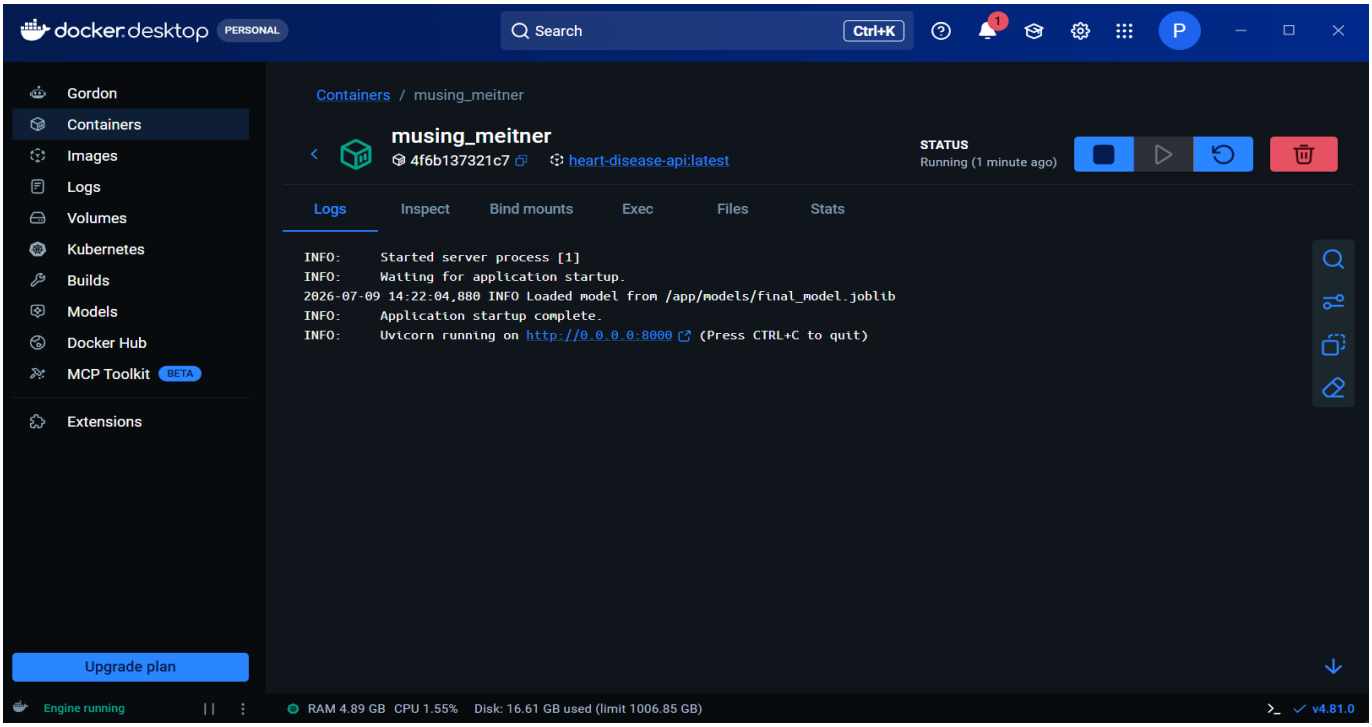
The Dockerfile builds an isolated container for the model serving API. The image installs all dependencies, runs the data pipeline and training step, and starts the FastAPI application using uvicorn. This satisfies the requirement that the model must run correctly in an isolated environment.

Docker Component	Role
python:3.13-slim	Base runtime image
requirements.txt	Installs dependencies
src/*.py	Runs data acquisition, EDA and training
api.main:app	Starts FastAPI service

Screenshots:







12. Kubernetes Deployment

Command used:

kubectl get nodes

Build Docker Image: docker build -t heart-disease-api:latest .

docker images

kubectl apply -f k8s\deployment.yaml

kubectl apply -f k8s\service.yaml

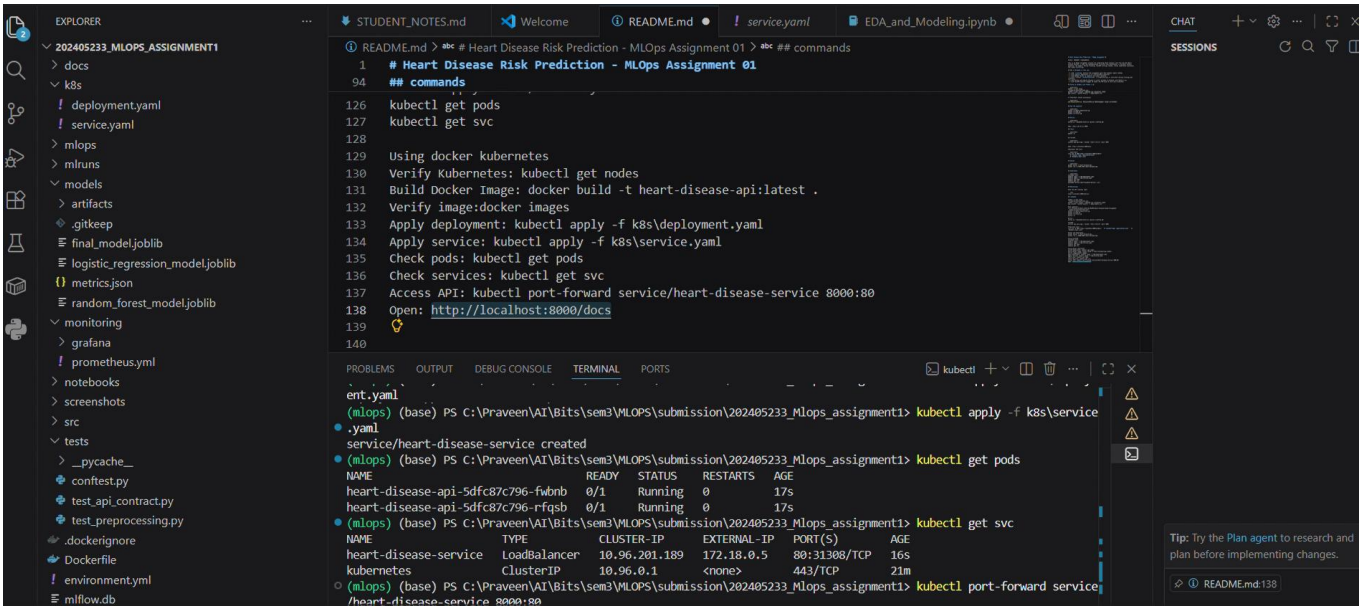
kubectl get pods

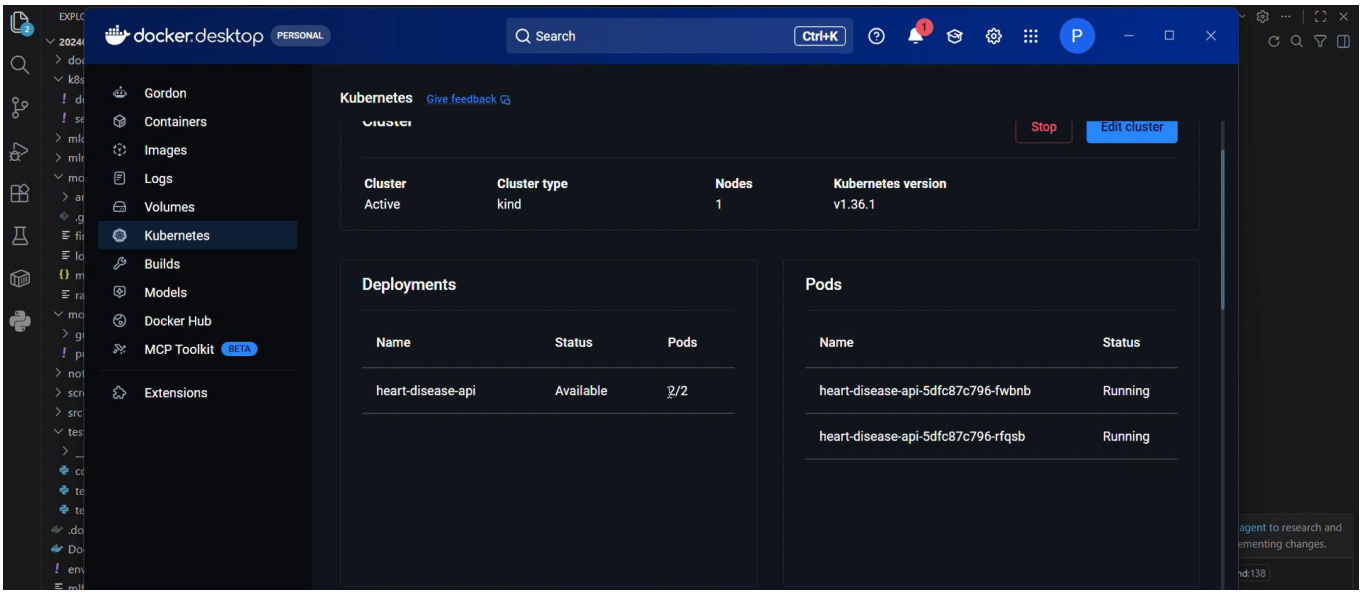
kubectl get svc

kubectl port-forward service/heart-disease-service 8000:80

Kubernetes manifests are included for local deployment using Docker Desktop Kubernetes. The deployment file creates two API replicas, while the service file exposes the application using a LoadBalancer type service.

File	Purpose
k8s/deployment.yaml	Defines pod replicas, container port and health probes.
k8s/service.yaml	Exposes the API service to local/cluster users.





13. Monitoring and Logging

Command used:

prometheus

docker run -d --name prometheus -p 9090:9090 prom/prometheus

docker run -d --name prometheus -p 9090:9090 prom/Prometheus

Graphana

docker run -d --name grafana -p 3000:3000 grafana/Grafana

Prometheus UI: http://localhost:9090

http://localhost:9090/targets

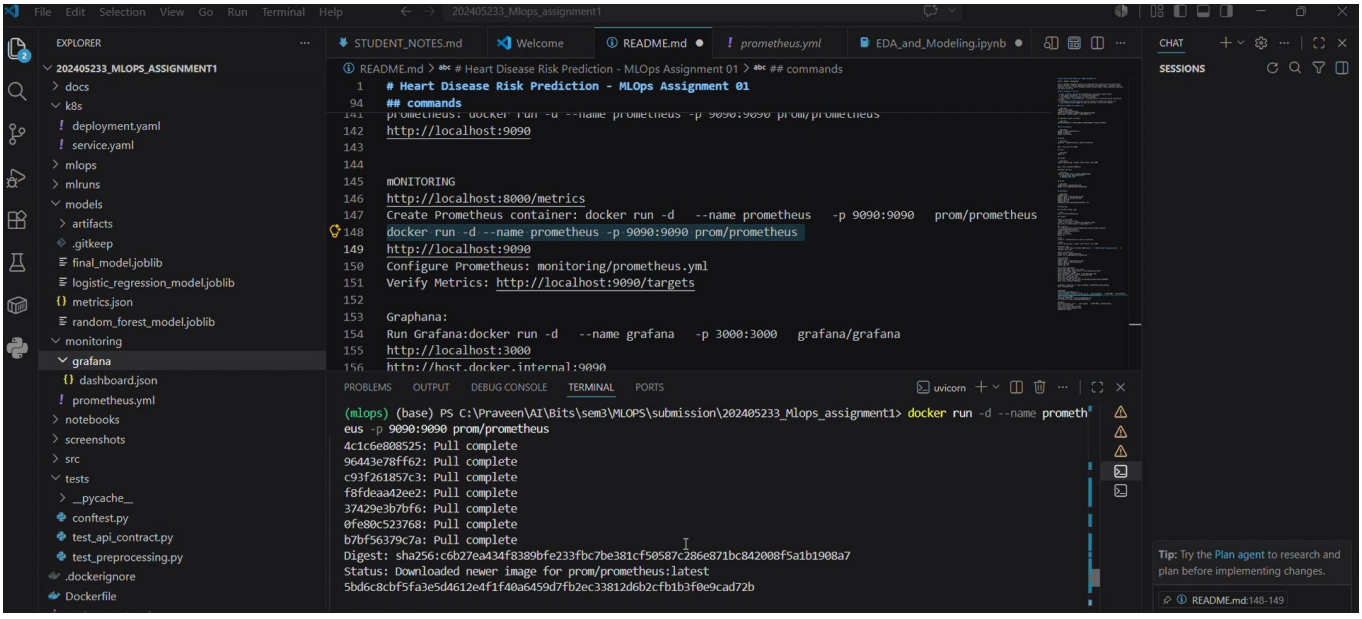
Graphana UI: http://localhost:3000

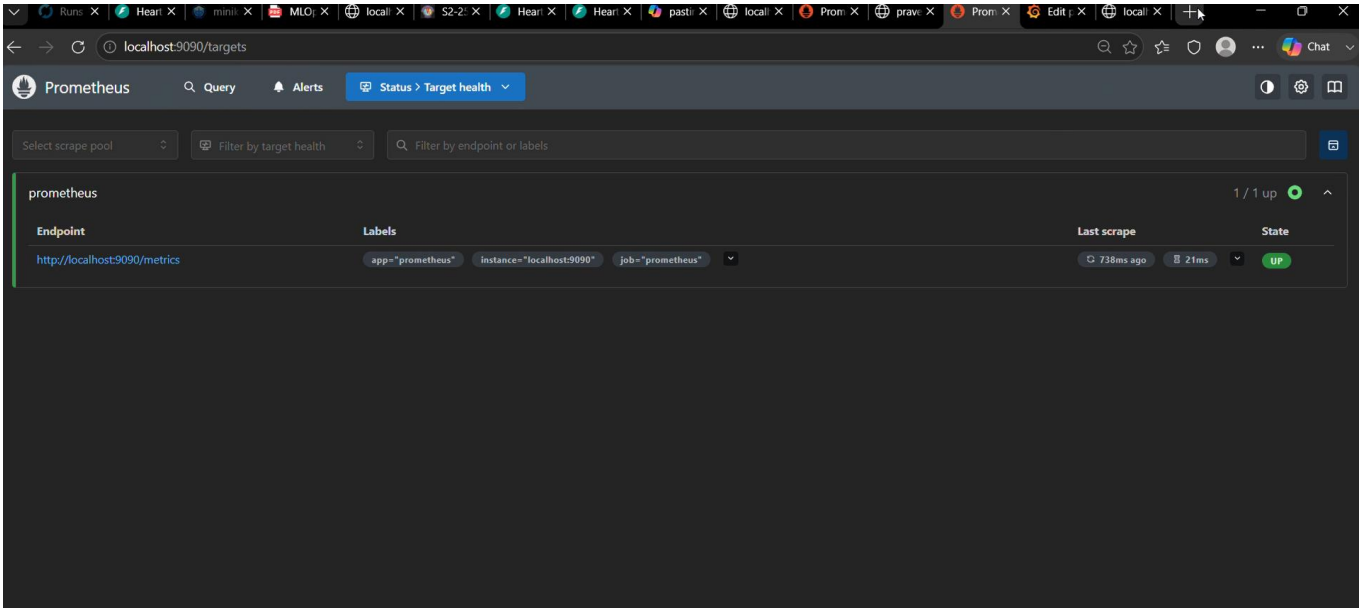
http://host.docker.internal:9090

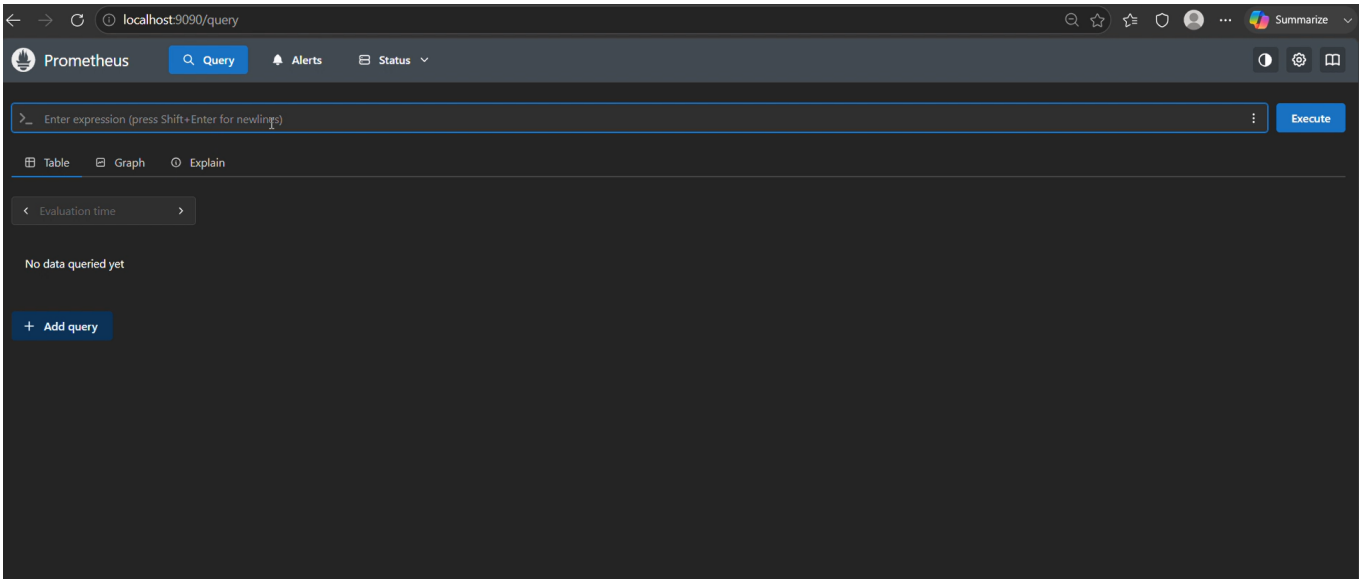
The API includes request logging and exposes Prometheus-compatible metrics at /metrics. The monitoring folder contains a Prometheus scrape configuration and a starter Grafana dashboard JSON. This demonstrates basic observability for API requests, prediction counts and request latency.

- heart_api_requests_total: counts API requests by endpoint, method and status.
- heart_api_predictions_total: counts predictions by class.
- heart_api_request_latency_seconds: records API request latency.

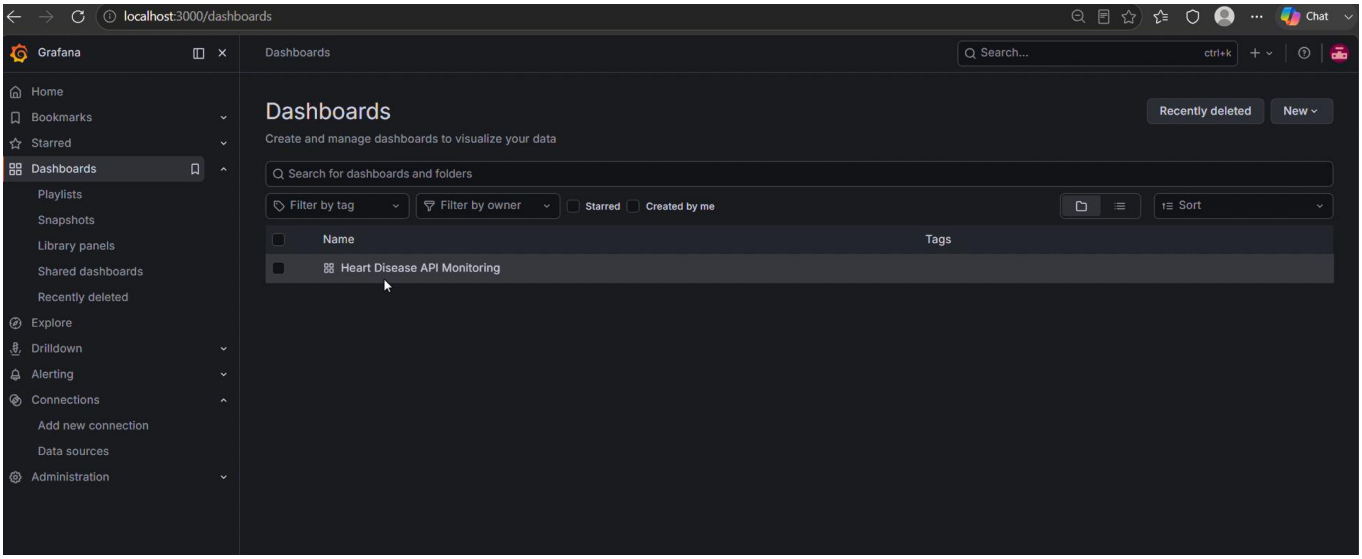
Prometheus:




localhost:9090/targets
Prometheus
Query
Alerts
Status > Target health
Select scrape pool
Filter by target health
Filter by endpoint or labels
prometheus
1 / 1 up
Endpoint
Labels
Last scrape
State
http://localhost:9090/metrics
app="prometheus"
instance="localhost:9090"
job="prometheus"
738ms ago
21ms
UP


localhost:9090/query
Prometheus
Query
Alerts
Status
Enter expression (press Shift+Enter for newlines)
Execute
Table
Graph
Explain
Evaluation time
No data queried yet
Add query

Graphana:


localhost:3000/dashboards
Grafana
Dashboards
Search...
Home
Bookmarks
Starred
Dashboards
Playlists
Snapshots
Library panels
Shared dashboards
Recently deleted
Explore
Drilldown
Alerting
Connections
Add new connection
Data sources
Administration
Dashboards
Create and manage dashboards to visualize your data
Recently deleted
New
Search for dashboards and folders
Filter by tag
Filter by owner
Starred
Created by me
Sort
Name
Tags
Heart Disease API Monitoring

13.1 Architecture Flow

```
UCI Dataset -> Cleaning/EDA -> Preprocessing Pipeline -> Model Training -> MLflow Tracking -> Joblib
Model -> FastAPI -> Docker -> Kubernetes -> Prometheus/Grafana
```

15. Submission Links

Step	Location/Remark
EDA plots	docs/figures/
MLflow runs	http://127.0.0.1:5000
API prediction	http://localhost:8000/docs
Tests	pytest -q
Docker proof	docker build/run
Kubernetes proof	kubectl get pods/svc
GitHub Actions	GitHub repository Actions tab
Demo video	Add recording link before upload

16. Conclusion

This project implements the complete MLOps lifecycle for a binary heart disease prediction model. It includes automated dataset acquisition, EDA, preprocessing, model tuning, experiment tracking, reproducible model packaging, API serving, automated testing, CI/CD, Docker containerization, Kubernetes deployment files and monitoring support. The design focuses on reproducibility and clear separation of responsibilities so that each stage can be run and validated independently.